

Windows: системные API и сбор телеметрии

Практический справочник для Windows Agent в AI PC Guardian

WinAPI	CPU / RAM	Processes	Named Pipe
--------	-----------	-----------	------------

Что даст этот справочник

После чтения ты сможешь выбрать правильный WinAPI, понять его результат, безопасно закрыть ресурсы, собрать основные метрики и встроить их в текущий pipeline проекта.

Платформа: Windows 10/11 | Редакция: 01.09.2026 | Язык примеров: C++17

Материал привязан к архитектуре AI PC Guardian. Это учебная выжимка, а не замена официальной документации API.

Содержание

1. Что именно строит AI PC Guardian	3
Термины: переменные окружения и системные метрики	3
Контракт одного измерения	3
2. Ментальная модель WinAPI	4
3. Переменные окружения и системные каталоги	4
4. CPU: почему нужны два снимка	5
5. Оперативная память	6
6. Uptime, диск и файловая система	6
7. Сеть	6
8. Процессы	7
9. Идентификаторы устройства и ОС	8
10. Named Pipe: локальный IPC текущего проекта	8
11. Консольный процесс и Windows Service	9
12. Ошибки, логи и диагностика падений	9
13. Рекомендуемая структура кода	9
14. Порядок реализации Windows MVP	10
15. Чек-лист code review	10
16. Официальные источники	10

1. Что именно строит AI PC Guardian

Клиент под Windows - это локальный агент. Он регулярно читает измеримые факты у операционной системы, приводит их к общей модели `guardian::common`, сохраняет последний снимок, сериализует сообщения через Protobuf и отправляет их локальному интерфейсу или серверу. Сетевой сервер уже решает, что хранить, агрегировать и передавать модели.



Короткий ответ на главный вопрос

На Linux не надо запускать `top`, `free`, `df` и разбирать их текст. Для MVP агент читает стабильные kernel-интерфейсы `/proc`, `/sys` и системные вызовы. Утилиты оставляем только для ручной диагностики. Python и LLM находятся на сервере; системный агент остается предсказуемым C++-процессом.

Термины: переменные окружения и системные метрики

Это две разные темы. Переменные окружения - строки конфигурации процесса: путь к каталогу данных, адрес сервера, уровень логирования. Системные метрики - CPU, RAM, диск, сеть, процессы и uptime, которые читаются через API ОС. В справочнике разобраны обе темы, но секреты и API-ключи не должны попадать в клиентскую телеметрию или логи.

Слой	Ответственность	Не должен делать
Agent / C++	Собирать факты, валидировать диапазоны, ставить timestamp, сериализовать	Интерпретировать диагноз с помощью LLM
Backend	Аутентификация, прием, хранение, агрегация, rate limit	Доверять любому полю клиента без проверки
AI service / Python	Редакция контекста, prompt, вызов модели, проверка ответа	Получать полный дамп системы без минимизации данных
GUI	Показывать состояние, ошибки и рекомендации	Блокировать поток сборщика

Контракт одного измерения

- Считывай счетчики дважды, если метрика является скоростью или процентом: CPU, bytes/sec, packets/sec.
- Храни сырые величины в целых числах и явно фиксируй единицы: bytes, milliseconds, percent.
- Отмечай источник и ошибку отдельным полем; 0 не должен означать одновременно реальный ноль и сбой.
- Используй monotonic clock для интервалов и system clock для timestamp события.
- Сборщик не бросает исключение наружу на обычном отказе доступа: он возвращает ошибку и продолжает собирать другие метрики.

2. Ментальная модель WinAPI

WinAPI выглядит непривычно из-за C-интерфейса. Большинство функций принимает указатели на структуры, возвращает `BOOL`, `HANDLE` или числовой код, а подробную ошибку нужно читать через `GetLastError()` сразу после неудачного вызова. Windows-строки обычно UTF-16: используй версии функций с суффиксом `W` и `std::wstring`.

Тип / правило	Что означает	Что делать в C++
<code>HANDLE</code>	Ссылка на kernel object или псевдообъект	Проверить <code>invalid value</code> и закрыть правильной функцией
<code>BOOL</code>	Ноль - ошибка, ненулевое - успех	При нуле сразу сохранить <code>GetLastError()</code>
<code>DWORD</code>	32-битное беззнаковое значение	Не путать с размером буфера в байтах/символах
<code>FILETIME</code>	64-битное число интервалов по 100 ns	Склеить <code>high/low</code> в <code>uint64_t</code>
<code>...W</code>	Unicode UTF-16 вариант API	Предпочитать <code>W</code> , не зависеть от макроса <code>UNICODE</code>

Минимальная проверка `BOOL`. В production-collector вместо исключения удобно вернуть `std::expected` или собственный `Result`.

```
#include <windows.h>
#include <system_error>

void check_win32(BOOL ok, const char* operation) {
    if (ok) return;
    const DWORD code = GetLastError(); // прочитать сразу
    throw std::system_error(
        static_cast<int>(code),
        std::system_category(), operation);
}
```

RAII обязателен

`HANDLE` от `CreateToolhelp32Snapshot`, `CreateFileW`, `CreateNamedPipeW` закрывается через `CloseHandle`. Память от `GetIfTable2` освобождается через `FreeMibTable`. Один универсальный deleter подходит не всем ресурсам.

3. Переменные окружения и системные каталоги

Переменная окружения принадлежит процессу и наследуется дочерними процессами. Она удобна для локальной настройки: `GUARDIAN_SERVER_ADDRESS`, `GUARDIAN_LOG_LEVEL`, `GUARDIAN_DATA_DIR`. Для пользовательских каталогов не склеивай путь из `%USERPROFILE%`; спрашивай Known Folder API.

Двухпроходный шаблон: сначала узнать размер, потом выделить буфер.

```
#include <windows.h>
#include <optional>
#include <string>
#include <vector>

std::optional<std::wstring> env(std::wstring_view name) {
    const std::wstring key{name}; // гарантированный завершающий NUL
    SetLastError(ERROR_SUCCESS);
    const DWORD size = GetEnvironmentVariableW(key.c_str(), nullptr, 0);
    if (size == 0) {
        if (GetLastError() == ERROR_ENVVAR_NOT_FOUND) return std::nullopt;
        return std::wstring{}; // переменная существует, но пуста
    }

    std::vector<wchar_t> buffer(size);
    const DWORD written = GetEnvironmentVariableW(
        key.c_str(), buffer.data(), static_cast<DWORD>(buffer.size()));
    if (written == 0 || written >= buffer.size()) return std::nullopt;
    return std::wstring(buffer.data(), written);
}
```

- `GetEnvironmentVariableW`: получает значение одной переменной текущего процесса.
- `ExpandEnvironmentStringsW`: раскрывает строки вроде `%LOCALAPPDATA%\AI_PC_Guardian`.
- `SHGetKnownFolderPath(FOLDERID_LocalAppData, ...)`: правильный путь для данных пользователя; результат освобождается `CoTaskMemFree`.
- Системный Windows Service может работать под другим аккаунтом и видеть другое окружение. Критичную конфигурацию читай из явного `config`-файла или `service parameters`.
- API-ключ провайдера LLM хранится только на backend. Клиенту он не нужен.

4. CPU: почему нужны два снимка

`GetSystemTimes` возвращает накопленные времена `idle`, `kernel` и `user` с момента запуска системы. Это не готовый процент. Берутся два снимка, вычисляются дельты и только после этого доля занятого времени. `Idle` уже включен в `kernel`, поэтому его надо вычесть ровно один раз.

```
struct CpuTimes { uint64_t idle{}, kernel{}, user{}; };

uint64_t as_u64(FILETIME value) noexcept {
    ULARGE_INTEGER n{};
    n.LowPart = value.dwLowDateTime;
    n.HighPart = value.dwHighDateTime;
    return n.QuadPart;
}

std::optional<CpuTimes> read_cpu_times() {
    FILETIME idle{}, kernel{}, user{};
    if (!GetSystemTimes(&idle, &kernel, &user)) return std::nullopt;
    return CpuTimes{as_u64(idle), as_u64(kernel), as_u64(user)};
}

double cpu_percent(const CpuTimes& a, const CpuTimes& b) {
    const auto idle = b.idle - a.idle;
    const auto total = (b.kernel - a.kernel) + (b.user - a.user);
    if (total == 0 || idle > total) return 0.0;
    return 100.0 * static_cast<double>(total - idle) / total;
}
```

Интервал измерения

Для твоего `WindowsMetricCollector` разумный `cpu_sample_interval` - 250-1000 ms. Не делай `sleep_for` на GUI-потоке. Сборщик может спать в собственном worker thread, как `TelemetryRunner`.

5. Оперативная память

Для общей физической памяти подходит `GlobalMemoryStatusEx`. Перед вызовом обязательно записать `dwLength = sizeof(MEMORYSTATUSEX)`. Поле `ullAvailPhys` - доступная память, `ullTotalPhys` - общий объем.

```
struct MemorySample { uint64_t total_bytes; uint64_t available_bytes; };

std::optional<MemorySample> read_memory() {
    MEMORYSTATUSEX status{};
    status.dwLength = sizeof(status);
    if (!GlobalMemoryStatusEx(&status)) return std::nullopt;
    return MemorySample{
        static_cast<uint64_t>(status.ullTotalPhys),
        static_cast<uint64_t>(status.ullAvailPhys)};
}
```

В общую модель отправляй `total` и `available`, а `used` вычисляй как `total - available`. Это сохраняет исходные данные и уменьшает риск разных трактовок в клиентах.

6. Uptime, диск и файловая система

Метрика	WinAPI	Комментарий
Uptime	<code>GetTickCount64</code>	Миллисекунды с запуска; monotonic, не timestamp
Свободное место	<code>GetDiskFreeSpaceExW</code>	Передавай корень volume или путь внутри него
Тома	<code>FindFirstVolumeW / FindNextVolumeW</code>	Нужны, если показываем все volume
Тип диска	<code>GetDriveTypeW</code>	Removable, fixed, remote и т.д.
Disk I/O	PDH или storage APIs	Отдельный этап; это счетчики, нужны дельты

```
struct DiskSpace { uint64_t total; uint64_t available_to_user; };

std::optional<DiskSpace> read_disk_space(const wchar_t* path) {
    ULARGE_INTEGER available{}, total{}, free_total{};
    if (!GetDiskFreeSpaceExW(path, &available, &total, &free_total)) {
        return std::nullopt;
    }
    return DiskSpace{total.QuadPart, available.QuadPart};
}

const uint64_t uptime_ms = GetTickCount64();
```

7. Сеть

`GetIfTable2` возвращает таблицу `MIB_IF_ROW2`: имя интерфейса, состояние, MTU, входные и выходные octets/packets. Таблица выделяется системой и освобождается `FreeMibTable`. Для скорости снова нужны два снимка: `(bytes2 - bytes1) / elapsed_seconds`.

```
#include <iphlpapi.h>

void inspect_interfaces() {
    PMIB_IF_TABLE2 table = nullptr;
    if (GetIfTable2(&table) != NO_ERROR) return;

    for (ULONG i = 0; i < table->NumEntries; ++i) {
        const MIB_IF_ROW2& row = table->Table[i];
        if (row.OperStatus != IfOperStatusUp) continue;
        const uint64_t rx = row.InOctets;
        const uint64_t tx = row.OutOctets;
        // сохранить счетчики вместе с InterfaceLuid
    }
    FreeMibTable(table);
}
```

Что считать стабильным ID интерфейса

Не используй отображаемое имя как ключ: пользователь может его изменить. Для сопоставления снимков используй `NET_LUID`/interface index, а имя оставляй для GUI.

8. Процессы

Tool Help snapshot дает список процессов в конкретный момент. Между перечислением и `OpenProcess` процесс может завершиться - это нормальная гонка, а не авария. Отказ `ACCESS_DENIED` для системного или защищенного процесса тоже нормален.

```
#include <tlhelp32.h>

void enumerate_processes() {
    HANDLE snapshot = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    if (snapshot == INVALID_HANDLE_VALUE) return;

    PROCESSENTRY32W entry{};
    entry.dwSize = sizeof(entry);
    if (Process32FirstW(snapshot, &entry)) {
        do {
            const DWORD pid = entry.th32ProcessID;
            const std::wstring exe = entry.szExeFile;
            // OpenProcess только при необходимости дополнительных полей
        } while (Process32NextW(snapshot, &entry));
    }
    CloseHandle(snapshot);
}
```

Нужно узнать	API	Ограничение
Список PID/имя	Tool Help snapshot	Снимок быстро устаревает
Полный путь	<code>QueryFullProcessImageNameW</code>	Нужен process handle
Working set	<code>GetProcessMemoryInfo</code>	Нужен <code>psapi.h</code>
CPU процесса	<code>GetProcessTimes</code>	Сравнить два снимка и нормализовать
Командная строка	Сложнее / WMI / NT API	Собирать только при ясной пользе и согласии

Приватность

Имена процессов уже чувствительны, а command line может содержать токены, URL и пути пользователя. Для MVP отправляй минимум; перед LLM применяй allowlist полей и редактирование секретов.

9. Идентификаторы устройства и ОС

Для отображаемого имени можно использовать `GetComputerNameExW`. Но `hostname` меняется и не подходит как первичный ключ. Создай случайный UUID при первом запуске, запиши его в каталог данных приложения и отправляй как `DeviceId`. Это проще, переносимо и меньше похоже на fingerprinting железа.

- `DeviceId`: сгенерированный приложением UUID, устойчивый между перезапусками.
- `EventId`: новый UUID на каждое событие; обеспечивает идемпотентность приема.
- `hostname`: метка для GUI, не доверенный идентификатор.
- Версия ОС: используй Version Helper APIs/manifest; не парси строку `ver`.
- Версия агента: генерируется CMake из `version.hpp.in`.

10. Named Pipe: локальный IPC текущего проекта

Именованный pipe - канал обмена между процессами. Сервер создает `\\.\pipe\AI_PC_Guardian`, ждет клиента через `ConnectNamedPipe`, затем читает/пишет `ReadFile/WriteFile`. Byte-mode - поток байтов, поэтому границы Protobuf-сообщений надо задать самостоятельно. У вас уже выбран length-prefix: 4 байта длины + payload.



Схема текущего `IpcServer`: сначала валидируем размер, затем читаем ровно payload.

```

bool read_frame(HANDLE pipe, std::string& payload) {
    std::array<std::byte, 4> header{};
    if (!read_exact(pipe, header.data(), header.size())) return false;

    const uint32_t size = decode_u32_be(header);
    constexpr uint32_t kMaxFrame = 1024 * 1024;
    if (size == 0 || size > kMaxFrame) return false;

    payload.resize(size);
    return read_exact(pipe, payload.data(), payload.size());
}
  
```

- `ReadFile` может вернуть меньше запрошенного - `read_exact` обязан крутить цикл.
- Ограничивай максимальный frame до выделения памяти.
- Для остановки blocking I/O используй `CancelIoEx`, закрытие handle или overlapped I/O; один флаг сам по себе `ReadFile` не разбудит.
- `FlushFileBuffers` может ждать, пока клиент прочитает данные; не вызывай без понимания семантики.
- Каждое соединение и каждый pipe handle должны иметь ясного владельца.

Важно для релиза

Если `CreateNamedPipeW` получает `lpSecurityAttributes = nullptr`, default DACL может разрешать чтение более широкому кругу пользователей, чем нужно. Перед релизом задай явный security descriptor и проверь клиента по ожидаемому SID. Текущая реализация годится для локальной разработки, но ACL надо усилить.

11. Консольный процесс и Windows Service

Пока агент запускается как обычный процесс, его можно останавливать сигналом/console control handler и ждать worker thread. Для фонового production-режима Windows ожидает Service Control Manager: процесс вызывает `StartServiceCtrlDispatcher`, `ServiceMain` сразу регистрирует control handler и сообщает состояния `START_PENDING/RUNNING/STOP_PENDING/STOPPED`.

Сейчас	Позже как Service
<code>main()</code> создает runner и IPC	<code>ServiceMain()</code> создает runtime
Ctrl+C / console handler	SCM посылает <code>SERVICE_CONTROL_STOP</code>
Логи в файл/консоль	Логи в файл + Windows Event Log по необходимости
Окружение пользователя	Service account и явный config

12. Ошибки, логи и диагностика падений

- Сохраняй Win32 error code сразу после сбоя, затем форматируй его для лога.
- В каждом сообщении лога должны быть component, operation, device/event correlation id и error code.
- Не логируй Protobuf payload целиком, command line процессов, токены и персональные пути без редактирования.
- На старте логируй версию агента, конфигурацию без секретов и выбранные интервалы.
- Для необработанного падения следующий этап - minidump (`MiniDumpWriteDump`) и crash id; лог не всегда успевает записать последнюю строку.

```
auto sample = collector.collect_cpu();
if (!sample) {
    Logger::instance().error(
        "collector=cpu operation=GetSystemTimes error={}",
        sample.error().message());
} else {
    Logger::instance().debug(
        "collector=cpu usage_percent={:.2f}", sample->usage_percent);
}
```

13. Рекомендуемая структура кода

```
class WindowsMetricCollector final : public IMetricCollector {
public:
    Result<CpuMetric> collect_cpu(
        std::chrono::milliseconds sample_interval) override;
    Result<MemoryMetric> collect_memory() override;
    Result<DiskMetric> collect_disk(std::wstring_view path);
    Result<NetworkSnapshot> collect_network();
};

// Runner отвечает за расписание, collector - только за один сбор.
// Converter отвечает только за common <-> protobuf.
// Transport отвечает только за IPC/gRPC и timeout.
```

Компонент	Тестировать
Чистые формулы	CPU delta, network rate, used memory, overflow
Парсеры / conversion	Границы, неизвестный payload, round trip

Компонент	Тестировать
WinAPI adapter	Ручной smoke test и seam с fake adapter
Ipc framing	Partial read, zero/huge frame, disconnect, stop
Runner	Остановка, интервалы, частичный отказ collector/transport

14. Порядок реализации Windows MVP

- Закончить надежный length-prefixed IPC: partial read, лимит payload, graceful stop, ACL.
- Подключить CPU + RAM collector к [TelemetryRunner](#) и [LatestSnapshotStore](#).
- Добавить disk capacity, uptime и базовые network counters.
- Сделать GUI-клиент локального IPC: статус агента, свежесть snapshot, ошибки.
- Сделать установку/автозапуск, затем перенести host в Windows Service.
- Только после стабильной телеметрии подключать удаленный backend и LLM-рекомендации.

15. Чек-лист code review

- Все WinAPI return values проверены; error code не потерян между вызовами.
- Все HANDLE/table/buffer закрываются ровно один раз, включая ранние return.
- Для размеров использованы `size_t/uint64_t`, перед narrowing есть проверка.
- Строки приходят из W API и конвертируются в UTF-8 только на границе.
- Счетчики превращаются в rate/percent только по двум снимкам и реальному elapsed time.
- Отказ одной метрики не уничтожает весь snapshot.
- Поля телеметрии имеют единицы и validation; timestamp в UTC Unix milliseconds.
- IPC ограничивает frame, переживает partial I/O и проверяет локального клиента.
- Логи не содержат секретов; тесты покрывают формулы и framing.

16. Официальные источники

Ссылки ниже ведут на первичную документацию Microsoft. Читай страницу конкретной функции вместе с разделами Requirements и Remarks.

- [GetSystemTimes](#) - накопленные времена CPU
- [GlobalMemoryStatusEx](#) - состояние физической/виртуальной памяти
- [MEMORYSTATUSEX](#) - поля и обязательный dwLength
- [GetDiskFreeSpaceExW](#) - емкость и свободное место volume
- [GetIfTable2](#) - таблица сетевых интерфейсов
- [Tool Help snapshots](#) - снимки процессов, потоков и модулей
- [GetProcessMemoryInfo](#) - память процесса
- [GetProcessTimes](#) - времена процесса
- [Environment Variables](#) - модель окружения Windows
- [GetEnvironmentVariableW](#) - чтение переменной окружения
- [Named Pipes](#) - обзор локального IPC
- [Named Pipe operations](#) - read/write/connect/flush/disconnect
- [Named Pipe security](#) - ACL и права доступа
- [Service functions](#) - Service Control Manager API
- [ServiceMain](#) - жизненный цикл Windows Service